CH 01: Introduction

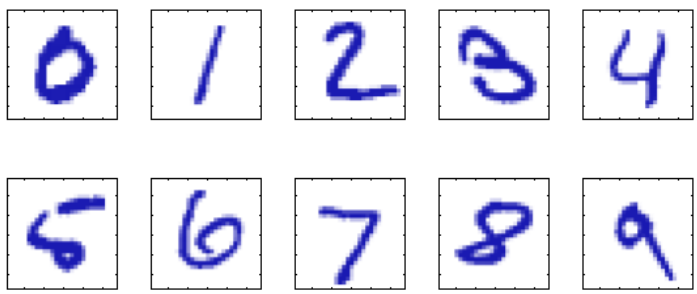
Contents

Pattern Recognition 2
Basic Key Terms 3
Polynomial functions 5
Polynomial Curve Fitting 7
Exercises 1 13
RMSE 14
Exercise 2 19
Regularization 19
Exercise 3 23
Bayesian Curve Fitting 23
Exercise 4 26

Pattern Recognition

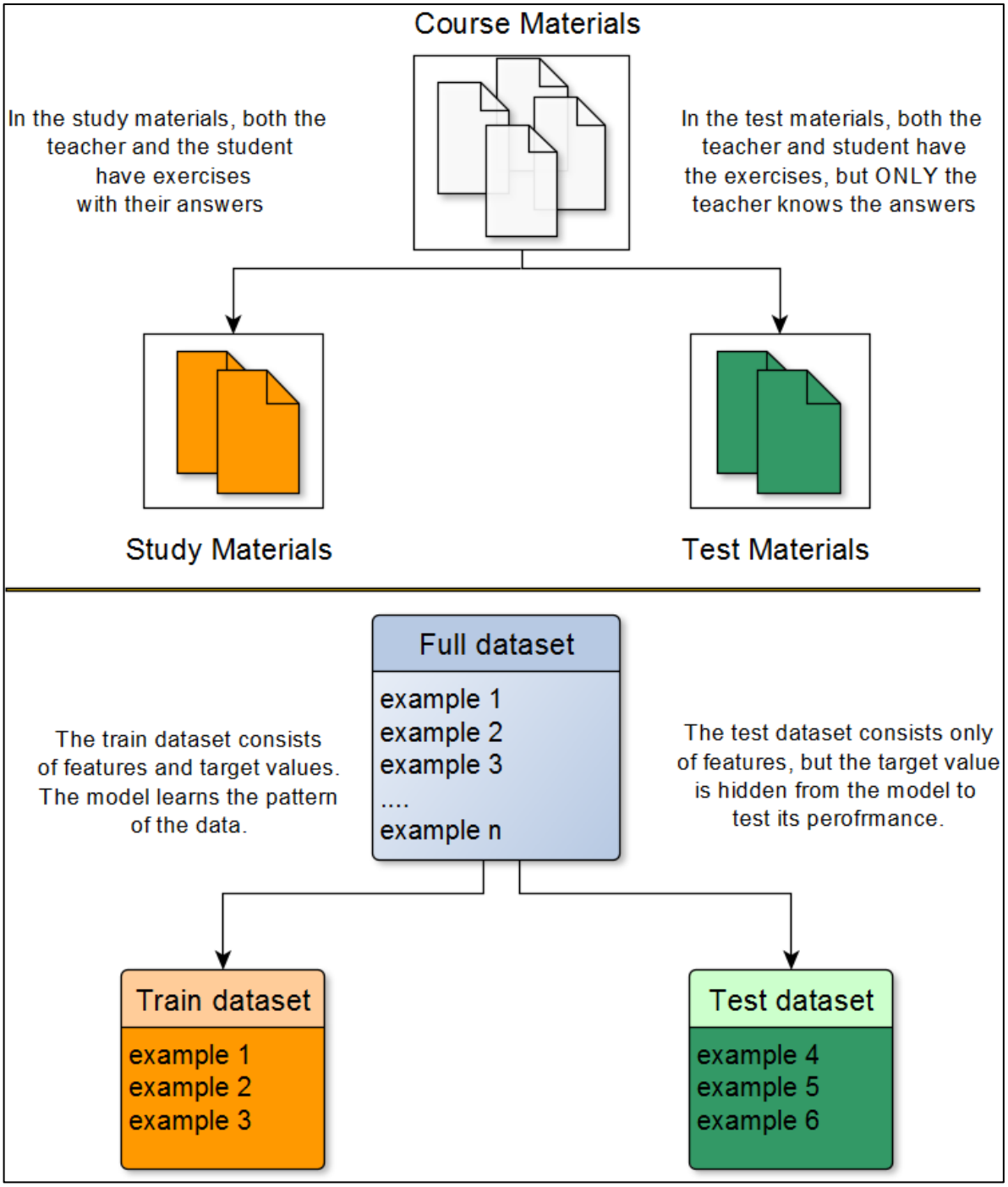
- **Pattern recognition** is the process of recognizing patterns by using a machine learning algorithm.
- Pattern recognition can be defined as
  - The classification of data based on knowledge already gained
  - Statistical information extracted from patterns and/or their representation.
- Consider the example of recognizing handwritten digits, illustrated in Figure 1.1. Each digit corresponds to a 28×28-pixel image and so can be represented by a vector x comprising 784 real numbers. The goal is to build a machine that will take such a vector x as input and that will produce the identity of the digit 0, . . ., 9
  - The vector x is called feature vector, that represents the characteristics of a certain object.

Figure 1.1 Examples of hand-written digits taken from US zip codes.



Basic Key Terms

- Dataset: A collection of data such that each row represents an example that has features and its corresponding target value.
    - If the dataset has features with target values (labels), then it is called labeled dataset.
    - If the dataset has only the features without target values (labels), then it is called unlabeled dataset.



- Dataset example:

Price	Floor space	Rooms	Lot size	Appartment	Row house	Corner house	Detached
250000	71	4	92	0	1	0	0
209500	98	5	123	0	1	0	0
349500	128	6	114	0	1	0	0
250000	86	4	98	0	1	0	0
419000	173	6	99	0	1	0	0
225000	83	4	67	0	1	0	0
549500	165	6	110	0	1	0	0
240000	71	4	78	0	1	0	0
340000	116	6	115	0	1	0	0

- Features: Input variables used in making predictions (deciding the target value).
- Training set: The subset of the dataset used to train (tune parameters of) a model.
- Target vector: Is the vector that holds the labels (values we want to predict) of the dataset.
- Training phase: The process of determining the ideal parameters comprising a model – that is, the phase that we try to learn pattern in the data set.
- Regression: The process of predicting continuous output (i.e., continuous number values). Example, predicting house price.
- Model: The representation of what a machine learning system has learned from the training data – simply, it is a function with learned parameters.

Polynomial functions

- A polynomial function is a function that involves only non-negative integer powers or only positive integer exponents of a variable in an equation.
- Types of Polynomial functions: (plot: https://www.desmos.com/calculator)

Type	Form
Zero Polynomial Function	$P(x) = a = ax^0$
Linear Polynomial Function	$P(x) = ax + b$
Quadratic Polynomial Function	$P(x) = ax^2 + bx + c$
Cubic Polynomial Function	$ax^3 + bx^2 + cx + d$
Quartic Polynomial Function	$ax^4 + bx^3 + cx^2 + dx + e$

- Zero Polynomial Function - Degree 0 (Constant Functions)
  - **Standard form**: $P(x) = a = a.x^0$, where a is a constant.
  - **Graph**: A horizontal line indicates that the output of the function is constant. It doesn't depend on the input.
  - **Example**: $y = 4$
- Linear Polynomial Functions - Degree 1, Linear Functions
  - **Standard form**: $P(x) = ax + b$, where a and b are constants. It forms a straight line.
  - **Graph**: Linear functions have one dependent variable and one independent which are x and y, respectively.
  - In the standard formula for degree 1, a represents the slope of a line, the constant b represents the y-intercept of a line.
  - **Example**: $y = 2x + 3$. Here $a = 2$ and $b = 3$.
- Quadratic Polynomial Functions - Degree 2, Quadratic Functions
  - **Standard form**: $P(x) = ax^2 + bx + c$, where a, b and c are constant.
  - **Graph**: A parabola is a curve with one extreme point called the vertex. A parabola is a mirror-symmetric curve where any point is at an equal distance from a fixed point known as Focus.
  - In the standard form, the constant 'a' represents the wideness of the parabola. As 'a' decrease, the wideness of the parabola increases. This can be visualized by considering the boundary case when $a = 0$,

the parabola becomes a straight line. The constant c represents the y-intercept of the parabola.

- Note: Whether the parabola is facing upwards or downwards, depends on the nature of a.
  - If $a > 0$, the parabola faces upward.
  - If $a < 0$, the parabola faces downwards.
- Example: $y = x^2 + 2x - 3$
- Example: $y = -x^2 - 2x + 3$

- **It is important to note that, training (learning a model) means that we try to find the optimal parameters (those are represented by $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}, \ldots \boldsymbol{etc}$.) that modifies the curve to the pattern of the data.**

Polynomial Curve Fitting

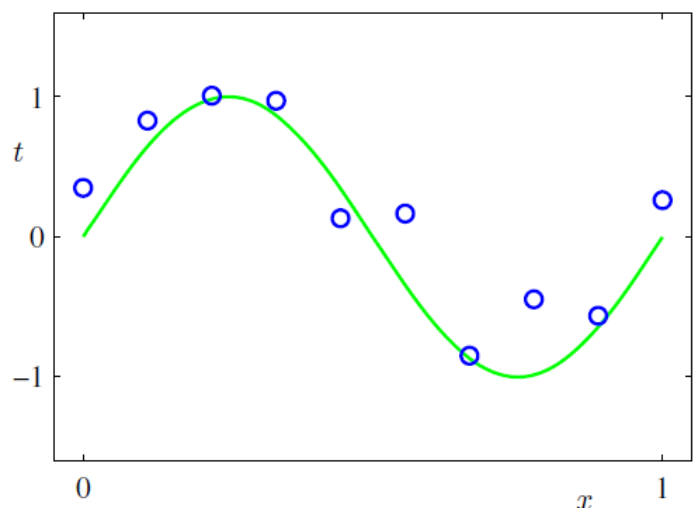
- We begin by introducing a simple regression problem, which we shall use as a running example throughout this chapter.
- Suppose we observe a real-valued input variable x and we wish to use this observation to predict the value of a real-valued target variable t.
    - We will consider an artificial example using synthetically generated data because we then know the precise process that generated the data for comparison against any learned model.
- The data for this example is generated from the function $\sin(2\pi x)$, with random noise included in the target value.
- The training set comprising N observations of x, written

$$x \equiv (x_1, \ldots, x_N)^T,$$

the corresponding observations of the values of t, denoted

$$t \equiv (t_1, \ldots, t_N)^T.$$

Figure 1.2 Plot of a training data set of $N = 10$ points, shown as blue circles, each comprising an observation of the input variable x along with the corresponding target variable t. The green curve shows the function $\sin(2\pi x)$ used to generate the data. Our goal is to predict the value of t for some new value of x, without knowledge of the green curve.



✓ Can we imagine different types of polynomials that resembles the pattern of the data? Try plotting the functions so that they are nearly as similar as possible to the original pattern which is $\sin(2\pi x)$.

- Degree 0.
- Degree 1.
- Degree 2.
- Degree 3

- Import libraries:
    - NumPy, Matplotlib, prml.preprocess, prml.linear
- Set random seed of the random generator to 1234

```python
# %%
import numpy as np
import matplotlib.pyplot as plt

from prml.preprocess import PolynomialFeature
from prml.linear import (
    LinearRegression,
    RidgeRegression,
    BayesianRegression
)

np.random.seed(1234)
```

- The function $create_dummy_data$ is used for generating feature vector, x, with its corresponding target values, t.
    - Parameters:
        - $func$: the function that describes pattern of the data.
        - $sample_size$: size of the generated data.
        - std: standard deviation for generating noise.
- The function $data_pattern$ defines the target values, t, of the feature values, x, $t(x) = \sin(2\pi x)$.

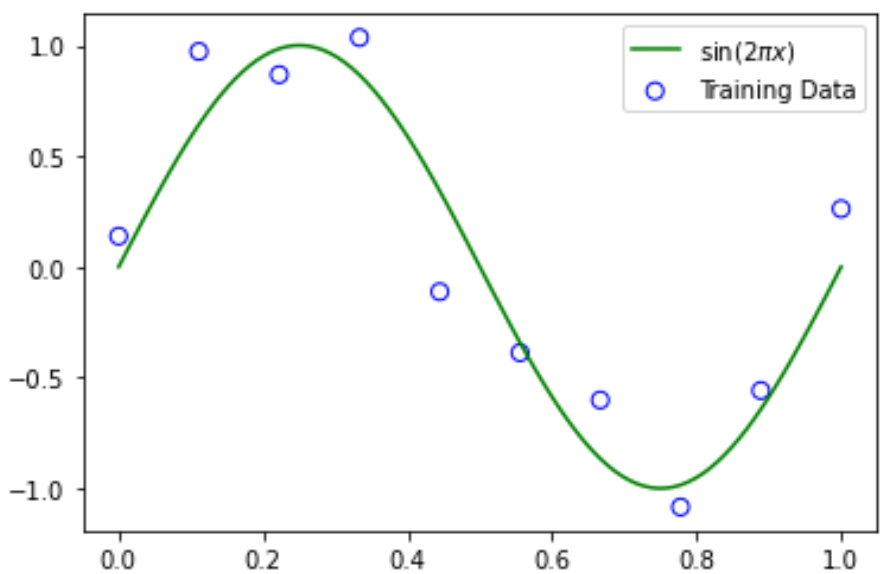
```python
def create_dummy_data(func, sample_size, std):
    # create data, x, that has values between 0 and 1
    x = np.linspace(0, 1, sample_size)
    # apply the pattern, func, to the feature values, x,
    # and add random noise to it
    t = func(x) + np.random.normal(scale=std,
                                   size=x.shape)
    return x, t


def data_pattern(x):
    return np.sin(2 * np.pi * x)
```

- Generate the training and test data

```
# %% GENERATE TRAINING AND TEST DATA
x_train, y_train = create_dummy_data(data_pattern,
                                     10, 0.25)

x_test = np.linspace(0, 1, 100)
y_test = data_pattern(x_test)
```

- Plot the training and test data

```
# %% PLOT THE DATA
plt.scatter(x_train, y_train, facecolor="none",
            edgecolor="b", s=50, label="Training Data")

# PLOT THE SIN FUNCTION (x_test and y_test)
plt.plot(x_test, y_test, c="green", label="$\sin(2\pi x)$")
plt.legend()
plt.show()
```



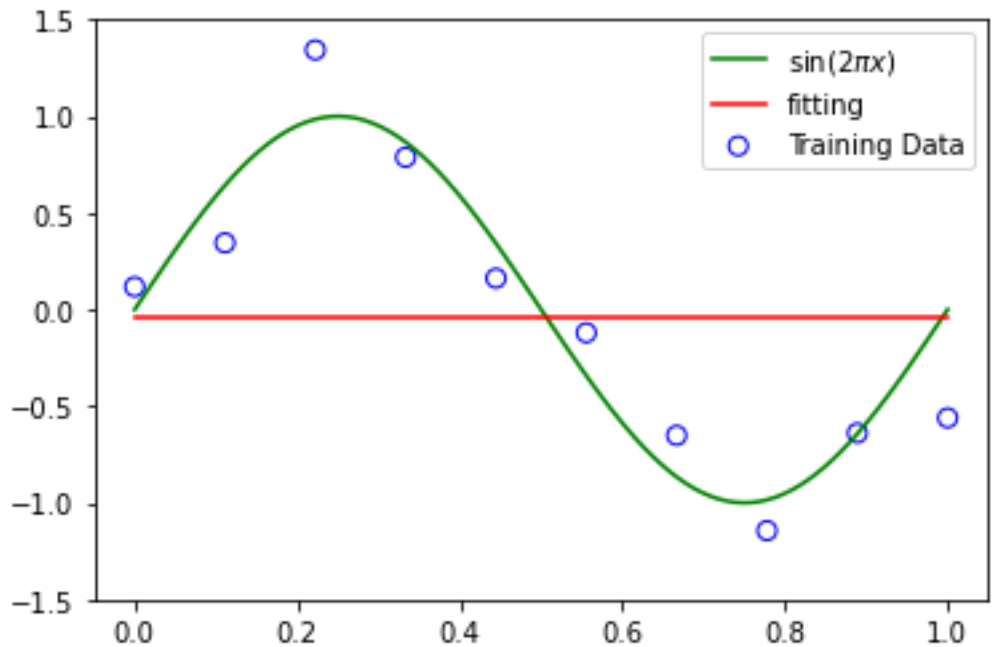
- The training data are plotted as blue scatters, the test data are plotted as curve, green line.
    - We see that the green line (test data) follows the same pattern exactly as the sin function.
    - But the scatters (training data) follow the pattern of the sin function, but not perfectly because of the noise added.

- Let's try fitting a regression model of degree 0.
    - This means that we build a model of the form $f(x) = a$, where x is our feature value, and a is a constant to learn.

```
# %% Fitting a model of degree 0
degree = 0
# Get Polynomial feature of degree 0
feature = PolynomialFeature(degree)
x_train_0 = feature.transform(x_train)
x_test_0 = feature.transform(x_test)

# Define regression model
model = LinearRegression()
# Let the model find the pattern of the data
#   (fittin the model to the data)
model.fit(x_train_0, y_train)

# Lets predict target values for the test data
y_0 = model.predict(x_test_0)
```

- Now, the variable y_0 holds the predictions of the model.
    - Compare the predicted values (y_0) in the memory with our actual valid target values (y_test). How similar are the values?
- To see how the model performs, plot the results.

```
# %% Plot the train and test data, and regression line

# plot train data
plt.scatter(x_train, y_train, facecolor="none",
            edgecolor="b", s=50, label="Training Data")

# plot test data with their original target values
plt.plot(x_test, y_test, c="green", label="$\sin(2\pi x)$")

# plot our prediction line (model perfromance)
plt.plot(x_test, y_0, c="red", label="fitting")

plt.ylim(-1.5, 1.5)
plt.legend()
plt.show()
```



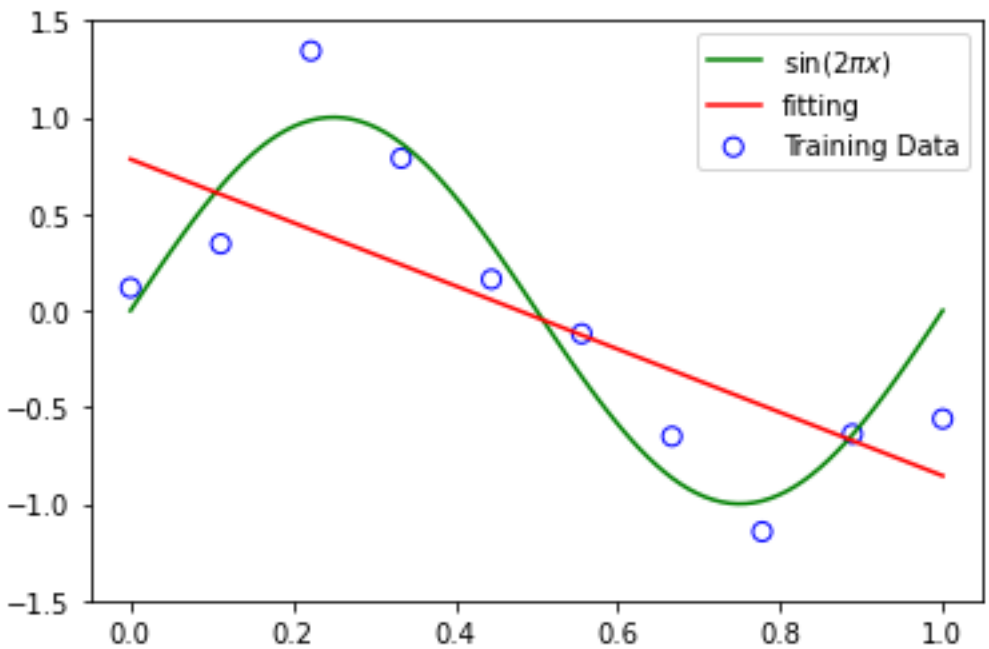
- Let's try fitting a regression model of degree 1.
  - This means that we build a model of the form $f(x) = ax + b$, where x is our feature value, and a is a constant to learn.

```
# %% Fitting a model of degree 1
degree = 1
# Get Polynomial feature of degree 1
feature = PolynomialFeature(degree)
x_train_1 = feature.transform(x_train)
x_test_1 = feature.transform(x_test)

# Define regression model
model = LinearRegression()
# Let the model find the pattern of the data
#   (fittin the model to the data)
model.fit(x_train_1, y_train)

# Lets predict target values for the test data
y_1 = model.predict(x_test_1)
```

  - Again, compare the model's prediction (y_1) with true target values (y_test).

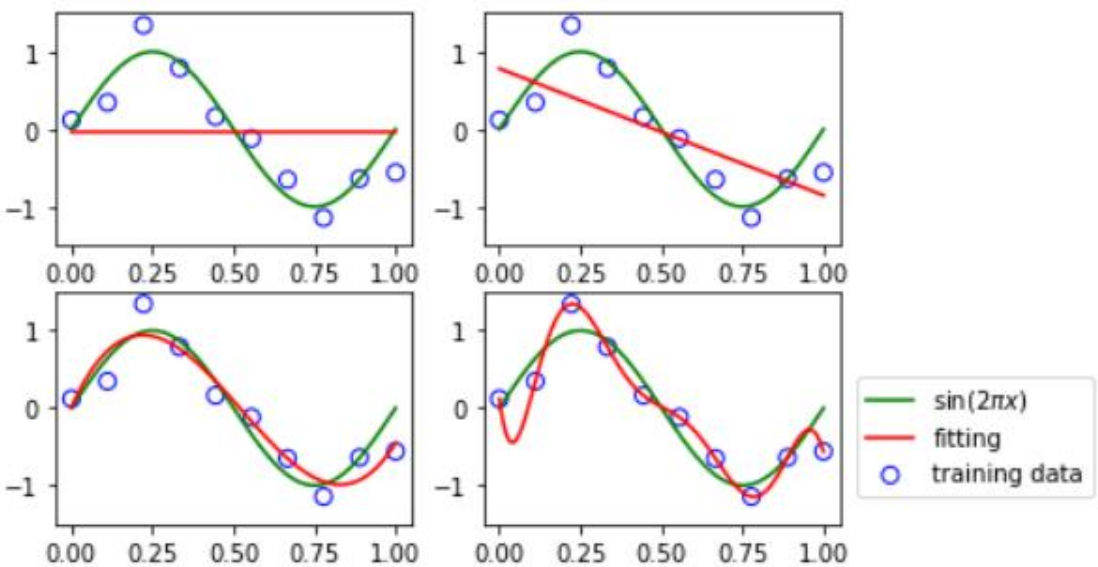
- Plot the model's performance.

```
# %% Plot the train and test data, and regression line

# plot train data
plt.scatter(x_train, y_train, facecolor="none",
            edgecolor="b", s=50, label="Training Data")

# plot test data with their original target values
plt.plot(x_test, y_test, c="green", label="$\sin(2\pi x)$")

# plot our prediction line (model perfromance)
plt.plot(x_test, y_1, c="red", label="fitting")

plt.ylim(-1.5, 1.5)
plt.legend()
plt.show()
```



Exercises 1

- For the data pattern of $\sin(2\pi x)$, create a model of degree 2, 3, and 4 and then plot the performance of each.
- It is practical in machine learning to plot various models within the same figure to make it easily to compare their performance. Plot the models of degree 0 to 3 in the same figure as below.



RMSE

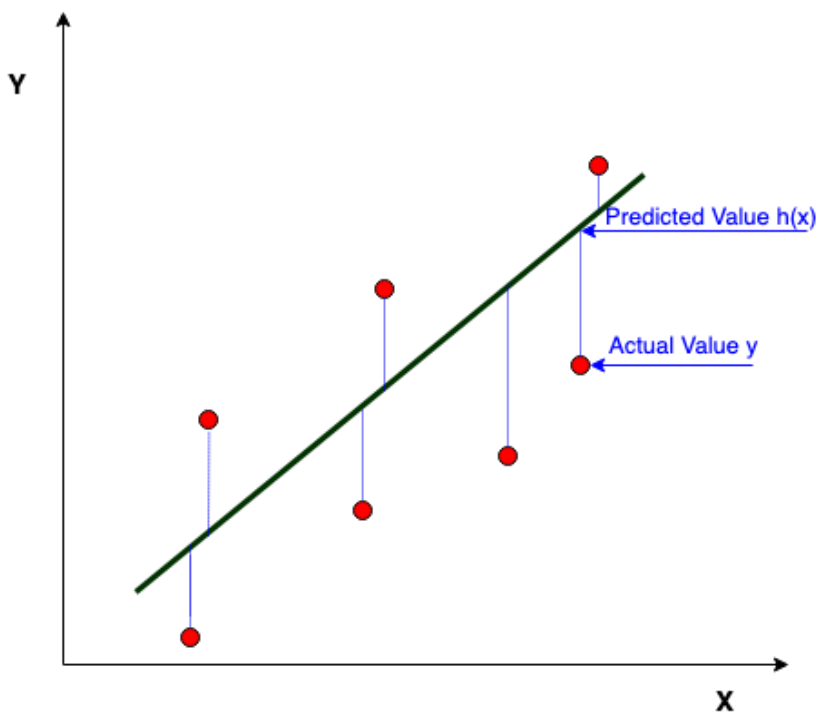
- Given some models of any degree, how we can tell that one model perform better that another model?
    - We use an error function that will measure the misfit (error) between our predictions (model's output) and the true values of the test set.
- We will use an error function called Root Mean Square Error (RMSE).
    - RMSE is given by the sum of the squares of the errors between the predictions for each data point and the corresponding target values.
- The RMSE formula is as follows:

$$RMSE = \sqrt{\sum_{i=1}^{n} \frac{(\hat{y}_i - y_i)^2}{n}}$$

$\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_n$ are predicted values

$y_1, y_2, \ldots, y_n$ are observed values

n is the number of observations



- Let's write the RMSE function:

```
#%% CREATE THE RMSE FUNCTION
def rmse(predicted_val, true_val):
    return np.sqrt(
        np.mean(
            np.square(
                predicted_val - true_val
                )
            )
        )

```

- Print the RMSE of the model of degree 0 and the model of degree 1
    - Which is better?

```
#%% PRINT RMSE VALUES OF THE TWO MODELS
rmse_0 = rmse(y_0, y_test)
rmse_1 = rmse(y_1, y_test)
print("The RMSE of a model of degree 0 is",
      rmse_0)
print("The RMSE of a model of degree 1 is",
      rmse_1)

```

- The smaller the value of the RMSE, the better its performance. The higher the value of the RMSE, the worse the model.
- Therefore, the goal of building machine learning models is to <u>optimize</u> the model to minimize the error function.

- Let's visualize how models' performance, both with training and test sets, changes in corresponds to the degree of the model.
    - We will create 10 models of degree 0 to 9.
    - For each model, we train it to the training set X_train with its target values y_train.
    - Test the model to the training set X_train.
    - Test the model to the testing set X_test.
    - Compute the RMSE of the model's prediction and the true target values of the test set y_test, and also, to the true target values of the training set y_train.
    - The last step is to plot the performance of each of the 10 models to compare their performance.

1. Create two lists, one for storing the error values of the train set and the other to store the error values on the test set.

```
#%% RMSE OF 10 MODELS
train_errors = []
test_errors = []

```

خلى باااالك من الحروف الكبيرة والصغيرة

2. Create a for loop from 0 to 10.
    a. Inside the loop, create feature values of degree i.
    b. Transform the training set x_train to another training set (X_train) with degree i.
    c. Transform the test set x_test to another test set (X_test) with degree i.

```
for i in range(10):
    feature = PolynomialFeature(i)
    X_train = feature.transform(x_train)
    X_test = feature.transform(x_test)

```

3. Inside the loop, create a linear regression model.
4. Train the model on the training data set.

```
    model = LinearRegression()
    model.fit(X_train, y_train)

```

5. let the model predict what are the target of values of the training set and the test set.

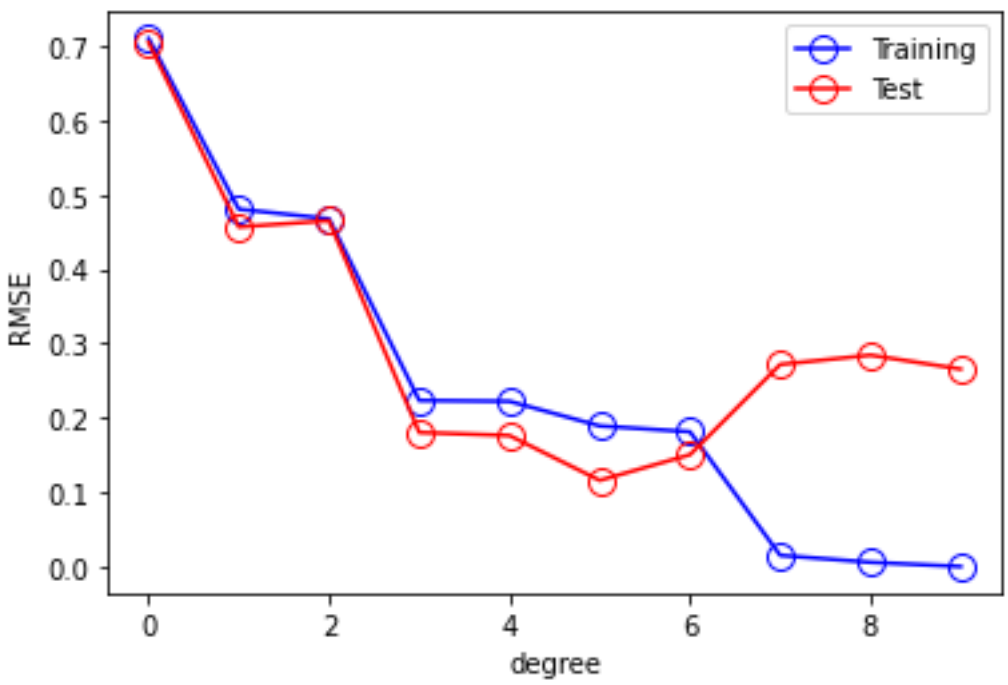
```
    # Model prediction on test set
    y = model.predict(X_test)
    # Model prediction on train set
    z = model.predict(X_train)

```

6. How good our model is? – compute the RMSE of both prediciton
    a. In this way, we evalute the model twice, one evaluation is based on the training set, which is the same set the model is trained on, and the other evalution is based on the test set, which is supposed to be unknown to the model.
7. Append the RMSE values in the corresponding list.

```
    rmse_train = rmse(z, y_train)
    rmse_test = rmse(y, y_test)

    train_errors.append(rmse_train)
    test_errors.append(rmse_test)
```

8. Finally, plot the results to visualize them.

```
#%% PLOT train_errors and test_errors
plt.plot(train_errors, 'o-',
         mfc="none", mec="b",
         ms=10, c="b", label="Training")
plt.plot(test_errors, 'o-',
         mfc="none", mec="r",
         ms=10, c="r", label="Test")
plt.legend()
plt.xlabel("degree")
plt.ylabel("RMSE")
plt.show()
```



Exercise 2

- Find the RMSE values of the models of degree 2, 3, and 4 you created in Exercise 1.
- Analyze the figure above of the RMSE train and RMSE test. Why the model performs very good (low RMSE) while we are increasing the degree until, but began to be very bad after degree 5? Meanwhile, the RMSE on the train set is very low (very good model).
- What is *overfitting* and *underfitting*?
- Why we test the model on both train set and test set?

Regularization

- Regularization helps prevent overfitting.
    - Overfitting is creating a model that matches the training data so closely that the model fails to make correct predictions on new data.
- Regularization adds a penalty term to the error function (MSE, in our case) to discourage the coefficients from reaching large values.
    - Our MSE function:

$$E(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\{y(x_n, \mathbf{w}) - t_n\}^2$$

where: N is the number of the training instances, $y(x_n, \boldsymbol{w})$ is the model's prediction on the input variable x_n given a specific set of weights (or coefficients, or parameters), t_n is the actual target value of the element x_n.

- By adding penalty term to the error function, it becomes:

$$\widetilde{E}(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

where $||\boldsymbol{w}||^2 \equiv \boldsymbol{w}^{\boldsymbol{T}}\boldsymbol{w} = w_0^2 + w_1^2 + w_2^2 + w_m^2$, such that m is the

degree of the function. The λ controls the amount of regularization applied to the model.

- To experiment the effect of the regularization term, we will use Ridge Regression model.
    - Ridge Regression is a model that is a variant of Linear Regression model that uses regularization within the error function.

- We will generate data with polynomial features of degree 9.

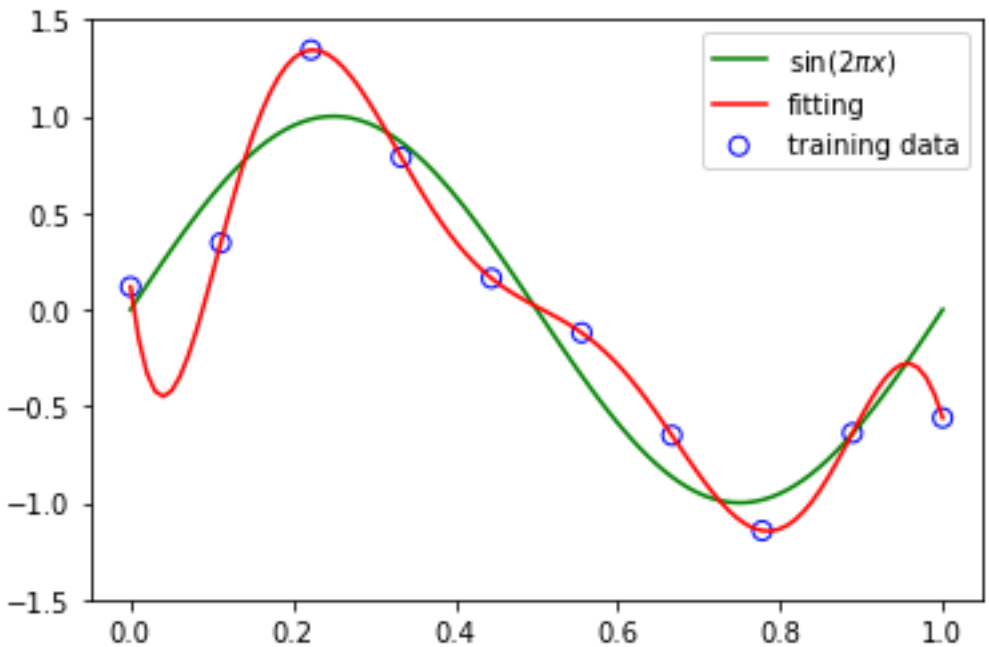
```
#%%
# features of degree 9
feature = PolynomialFeature(9)
# training set of degree 9
X_train = feature.transform(x_train)
# test set of degree 9
X_test = feature.transform(x_test)
```

- Fit a linear regression model to the training set.
- Then, test the model on the test dataset.

```
#%%
lin_reg_model = LinearRegression()
lin_reg_model.fit(X_train, y_train)
y_lin_reg = lin_reg_model.predict(X_test)

```

- Plot the results.

```
plt.scatter(x_train, y_train, facecolor="none",
            edgecolor="b", s=50, label="training data")
plt.plot(x_test, y_test, c="g", label="$\sin(2\pi x)$")
plt.plot(x_test, y_lin_reg, c="r", label="fitting")
plt.ylim(-1.5, 1.5)
plt.legend()
plt.annotate("M=9", xy=(-0.15, 1))
plt.show()
```

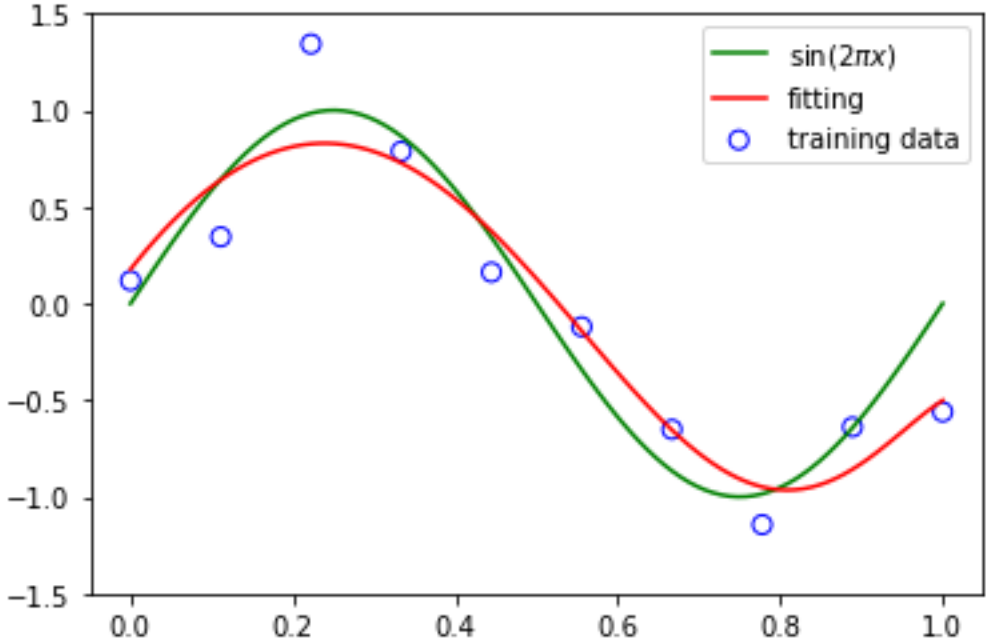


- Now, let's try Ridge regression with $\lambda = 1^{-3}$.
  - Fit a $RidgeRegression$ model to the training set.
  - Test the model's performance on the test set.

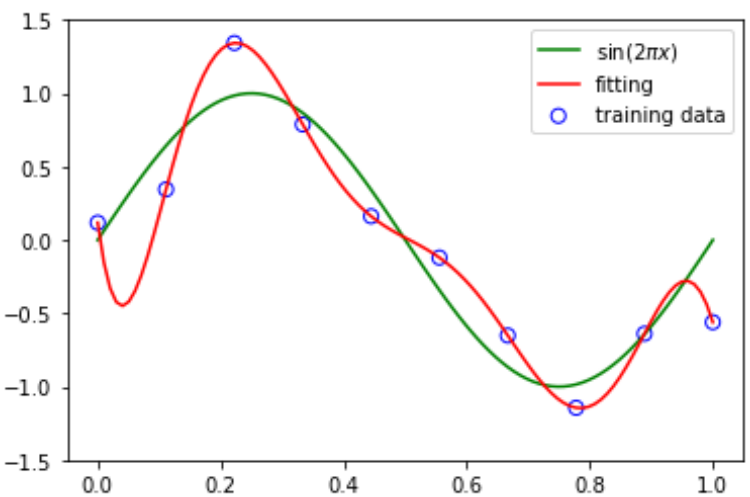
```
#%%
rid_reg_model = RidgeRegression(alpha=1e-3)
rid_reg_model.fit(X_train, y_train)
rid_reg_y = rid_reg_model.predict(X_test)

```

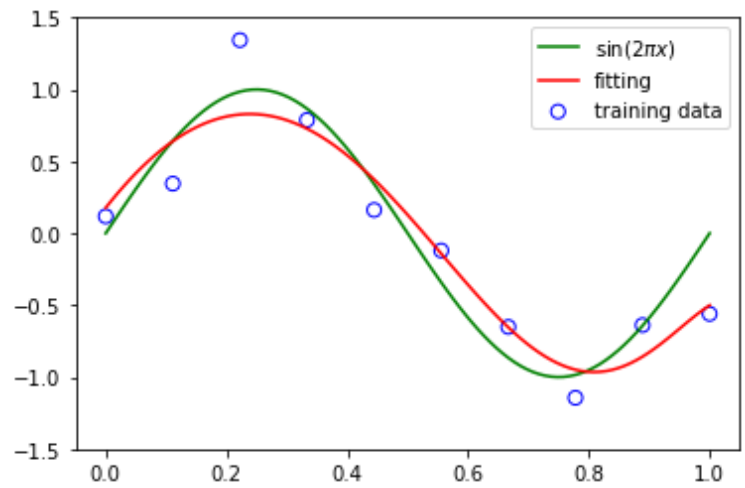
- Plot the results.

```
plt.scatter(x_train, y_train, facecolor="none",
            edgecolor="b", s=50, label="training data")
plt.plot(x_test, y_test, c="g", label="$\sin(2\pi x)$")
plt.plot(x_test, rid_reg_y, c="r", label="fitting")
plt.ylim(-1.5, 1.5)
plt.legend()
plt.annotate("M=9", xy=(-0.15, 1))
plt.show()
```



- Notice the difference between a regularized model and a non-regularized model. Which one is better? Why?





- Print the RMSE of each model.

```
#%%
rmse_lin_reg = rmse(y_lin_reg, y_test)
rmse_rid_reg = rmse(y_rid_reg, y_test)
print("The RMSE of a linear regression model is",
      rmse_lin_reg)
print("The RMSE of a ridge regrssion model is",
      rmse_rid_reg)
```

- ✓ Try different values of λ. What did you notice?
- ✓ set λ to a fixed value, 1^{-3} for example, try different values of model's degree. Is Ridge regression is better in all cases?
- ✓ View model's learned coefficients using $.w$ variable, which is a list of the weights of the model. [$print(rid_reg_model.w)$]

Exercise 3

- What are the common techniques of regularization?

Bayesian Curve Fitting

- In this section we will use another way of curve fitting that is based on Bayes theorem.
- Bayes Theorem describes the probability of an event, based on prior knowledge of conditions that might be related to the event.
    - For example, if the risk of developing health problems is known to increase with age, Bayes' theorem allows the risk to an individual of a known age to be assessed more accurately (by conditioning it on their age).
- Bayes theorem states the following:

$$\text{Posterior} = \text{Prior} * \text{Likelihood}$$

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

- Bayes theorem has 3 key elements:
    - Prior: Probability distribution representing knowledge or uncertainty of a data object prior or before observing it. [$P(A)$]
    - Posterior: Conditional probability distribution representing what parameters are likely after observing the data object. [$P(A|B)$]
    - Likelihood: The probability of an object falling under a specific category or class. [$P(B|A)$]

- For a dataset D and set of weights $\boldsymbol{w}$, Bayes' theorem takes the form

$$p(\mathbf{w}|\mathcal{D}) = \frac{p(\mathcal{D}|\mathbf{w})p(\mathbf{w})}{p(\mathcal{D})}$$

  Where:
  - $p(\boldsymbol{w})$, is the prior probability of the values of the weights before observing data.
  - $p(D, \boldsymbol{w})$, is the likelihood, which expresses how probable the observed data set is for different settings of the parameter vector $\boldsymbol{w}$.
  - $p(D)$, is the normalization constant, which ensures that the posterior distribution on the left-hand side is a valid probability density and integrates to one.
- The Bayesian formula can be expressed as:

$$p(\mathcal{D}) = \int p(\mathcal{D}|\mathbf{w})p(\mathbf{w})\,\mathrm{d}\mathbf{w}.$$

  or

$$\text{posterior} \propto \text{likelihood} \times \text{prior}$$

  or

$$p(\boldsymbol{w}|D) \propto p(D|\boldsymbol{w}) * p(\boldsymbol{w})$$

- Because our dataset D is composed of feature values $\boldsymbol{x}$ and target values $\boldsymbol{t}$. Thus, we can re-write the Bayesian formula as follows:

$$p(\mathbf{w}|\mathbf{x}, \mathbf{t}, \alpha, \beta) \propto p(\mathbf{t}|\mathbf{x}, \mathbf{w}, \beta)p(\mathbf{w}|\alpha).$$

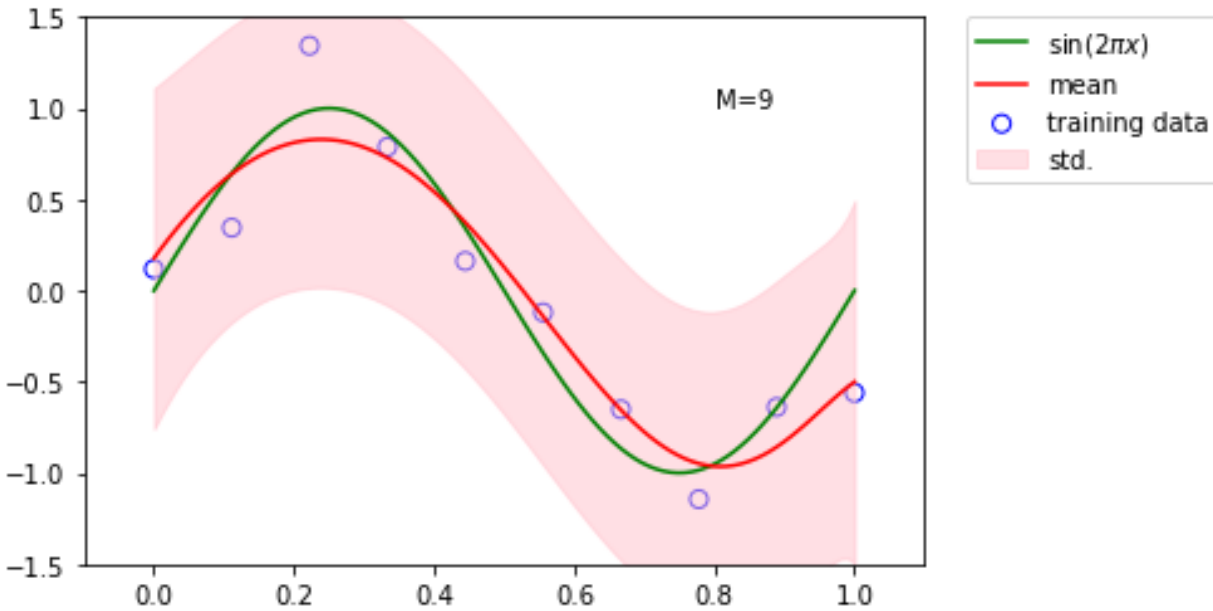
  Where:
  - D is decomposed into $\boldsymbol{x}$ (feature values) and $\boldsymbol{t}$ (target values)
  - β is a hyperparameter corresponds to the inverse of the variance.
  - α is a hyperparameter controls the distribution of the model's parameters.
  - The hyperparameters α and β can be used to determine a value for the regularization term λ by $\lambda = \alpha/\beta$.

- Let's fit a Bayesian model to our dataset
  - Define a Bayesian Regression model with $\alpha = 2^{-3}$ and $\beta = 2$.
  - Compute prediction values and the standard deviation of the prediction values.

```
#%%
bas_model = BayesianRegression(alpha=2e-3,
                               beta=2)
bas_model.fit(X_train, y_train)

bas_y, y_std = bas_model.predict(X_test,
                                 return_std=True)
```

  - Plot the training set, test set, and the standard deviation of the predictions.

```
#%%
plt.scatter(x_train, y_train, facecolor="none",
            edgecolor="b", s=50, label="training data")
plt.plot(x_test, y_test, c="g", label="$\sin(2\pi x)$")
plt.plot(x_test, bas_y, c="r", label="mean")
plt.fill_between(x_test, bas_y - y_std, bas_y + y_std,
                 color="pink", label="std.", alpha=0.5)
plt.xlim(-0.1, 1.1)
plt.ylim(-1.5, 1.5)
plt.annotate("M=9", xy=(0.8, 1))
plt.legend(bbox_to_anchor=(1.05, 1.), loc=2, borderaxespad=0.)
plt.show()
```



Exercise 4

- What are the advantages of using Bayesian linear regression? (i.e., what Bayesian regression is good for?)
- Compare the performance of Linear Regression model, Ridge Regression model, and Bayesian Regression model in terms of RMSE value for a dataset of degree 1 to 10. Plot the data and predictions for each model.